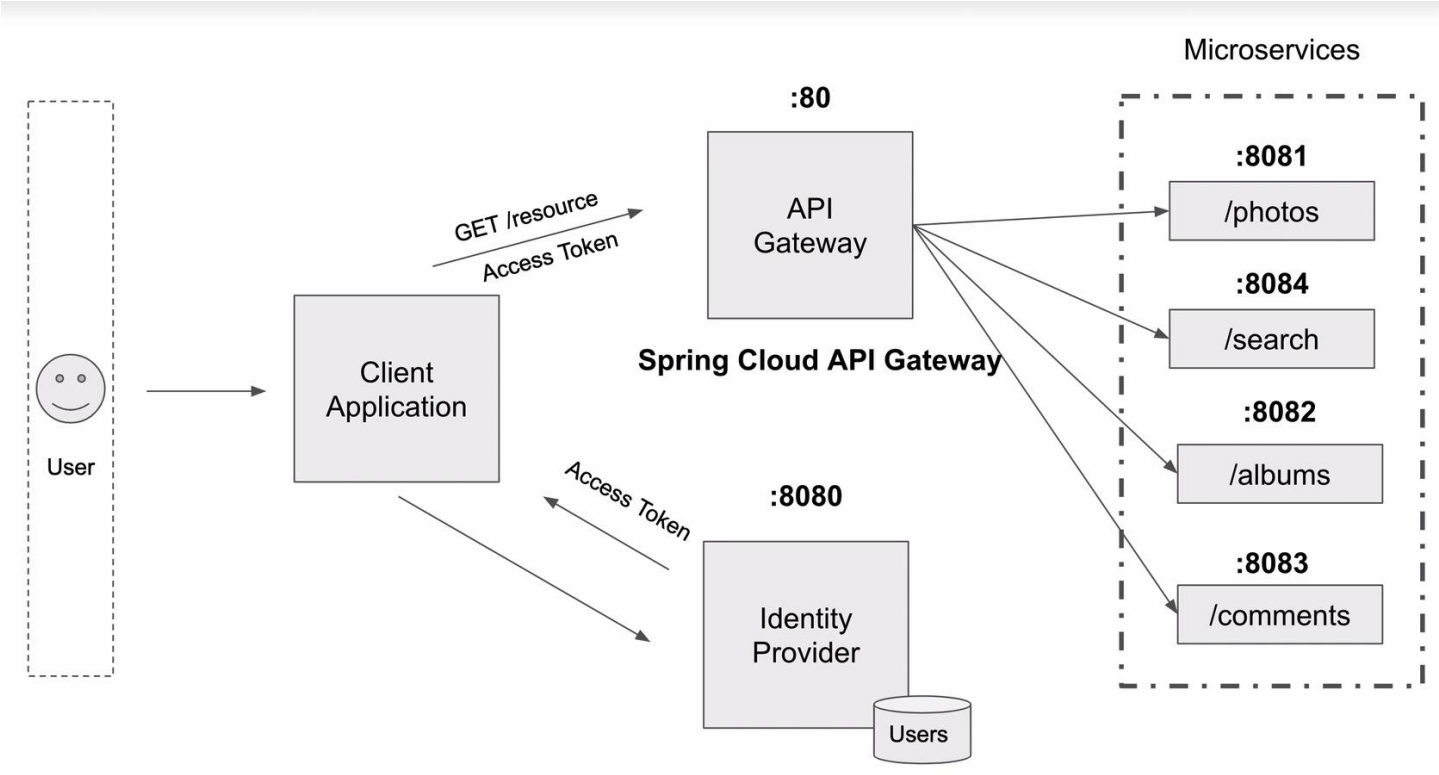
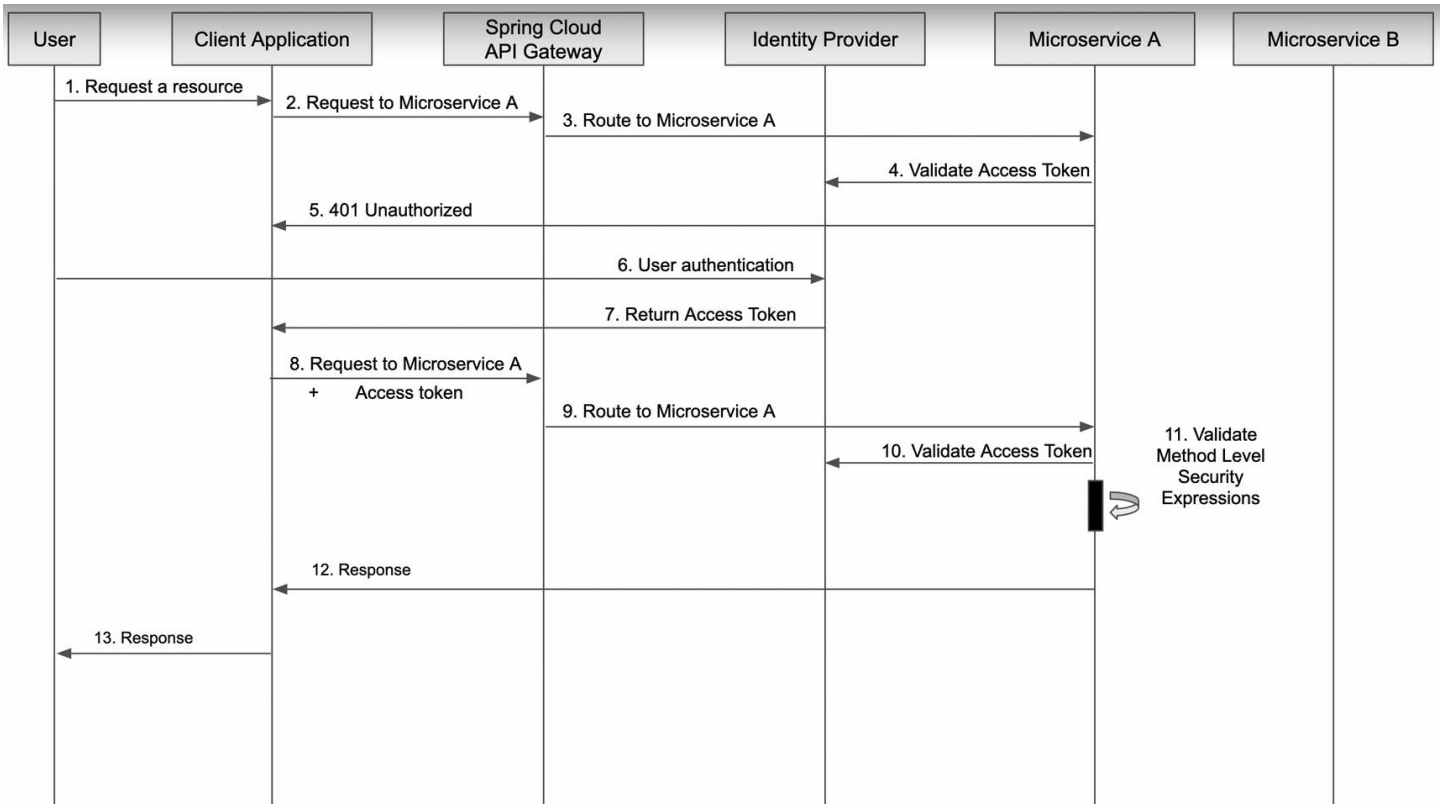
Creaet ApiGateWay



ដើម្បីអាចអោយយើងប្រើប្រាស់ Microservice Resource បានច្រើនយើងត្រូវបង្កើត ApiGateWay មួយដែលអាចអោយវាទៅយក MS របស់យើង



ដើម្បីអោយ client access to the resource server បានវាត្រូវធ្វើការ Request to the ApiGateWay first then the ApiGateWay will access to the Resource Server. In first time it will request token from the Client. If it not have the client will reques tot Auther Server to get the Token.

1- Create ApiGateWay Server

🔹 Dependencies

1️⃣ spring-boot-starter-web

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

✅ **Purpose:**

Adds **Spring MVC (Servlet-based web)** support.

📦 **Includes automatically:**

- Spring MVC
- Jackson (JSON)
- Embedded Tomcat
- Validation
- Logging

3️⃣ spring-boot-starter-actuator (Optional)

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

✅ **Purpose:**

Adds **monitoring & management endpoints**

Provides:

- `/actuator/health`
- `/actuator/info`
- `/actuator/metrics`
- `/actuator/env`
- etc.

💡 **Useful for:**

- Production monitoring
- Kubernetes readiness/liveness probes
- Observability tools

2️⃣ spring-cloud-starter-gateway-server-webmvc

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway-server-webmvc</artifactId>
</dependency>
```

✅ **Purpose:**

Adds **Spring Cloud Gateway (Servlet-based version)**.

🔥 **What it does:**

- Route incoming HTTP requests
- Forward to downstream services
- Apply filters (auth, logging, rate limiting)
- Rewrite paths
- Load balancing

⚠️ **Important:**

This is the **MVC version**, NOT the WebFlux version.

There are two gateway types:

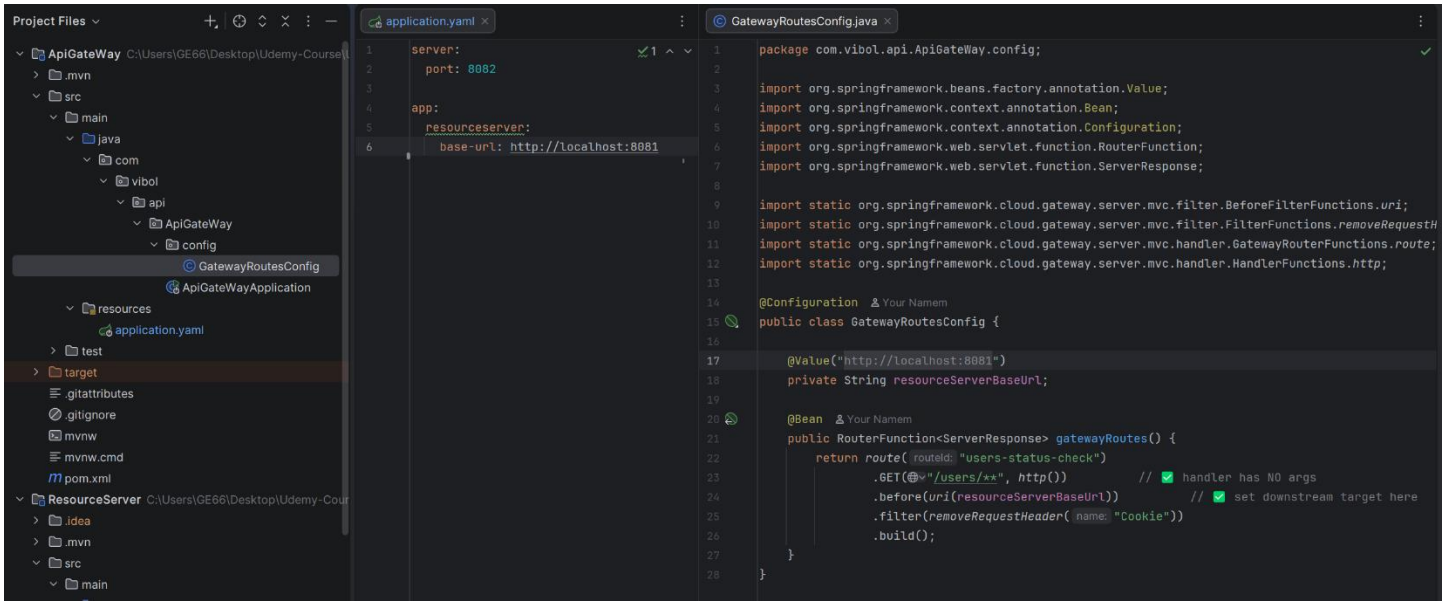
Version	Stack
`spring-cloud-starter-gateway`	Reactive (WebFlux)
`spring-cloud-starter-gateway-server-webmvc`	Servlet (Spring MVC)

ក្នុងនៅ Error យើងត្រូវអាចប្រើ ពីរ Webflux and MVC ដោយសារតែ Project នេះគេប្រើ Spring MVC ចឹងយើងត្រូវប្រើ MVC ដែល។ សម្រាប់ New Version Spring boot for the ApiGateway it will use spring cloud with Webflux.

- Spring Cloud MVC

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway-server-webmvc</artifactId>
</dependency>
```

2- Create configuration class in ApiGateWay



1) Your `application.yaml` (what it means)

```yaml
server:
  port: 8082

app:
  resourceserver:
    base-url: http://localhost:8081
```

Meaning

- Your **API Gateway app** runs on **port 8082**
- The **downstream resource server** (the service you forward to) is http://localhost:8081 ↗

So:

- Client calls **8082**
- Gateway forwards to **8081**

- ᨿᨾᨿ Client Called to the port 8082/users/status/check it will forward to port 8081

3) Full flow with your route

Your route:

```java
return route("users-status-check")
        .GET("/users/**", http())
        .before(uri(resourceServerBaseUrl))
        .filter(removeRequestHeader("Cookie"))
        .build();
```

What happens when a request comes in?

Example request (client → gateway)

```http
GET http://localhost:8082/users/123
Cookie: JSESSIONID=abc
```

Step-by-step inside the gateway

1. **Match the route**

- It matches because it is a **GET** and path starts with **/users/**

2. **Set downstream target**

- `.before(uri(resourceServerBaseUrl))`
- Since `resourceServerBaseUrl = http://localhost:8081`, the gateway rewrites destination to:

```http
GET http://localhost:8081/users/123
```

3. **Remove Cookie header**

4) Quick mental model (super clear)

Your system becomes:

- **Resource Server**: `http://localhost:8081`
- **API Gateway**: `http://localhost:8082`

And the rule is:

✅ `GET /users/**` on 8082 → forward to 8081
✅ remove `Cookie` header

3- Testing in Postman

- Client request token from Auth Server: port 8080

POST http://localhost:8080/realms/adminapps/protocol/openid-connect/token Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description	Bulk Edit
grant_type	password		
username	vibollong		
password	admin		
client_id	photo-app-code-flow-client		
client_secret	PYOtEeFuuKkCVohODQkp4L0PFKMQRBYQ		

Body Cookies Headers (9) Test Results (1/1) 200 OK 87 ms 4.04 KB Save Response

{} JSON Preview Visualize


```
{
    "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJvVFRlNXZEWUhiZkxlckV2M1U0YjFfQ2RJQUxXb1VUR2hRYjZUU0ZCelkwIn0.
        eyJleHAiOjE3NzI0NTYzNTgsImlhdCI6MTc3MjM2OTk1OCwianRpIjoib2ZydHJvOmVjMTBlNTUyLTBjYTgtMTMyMC11ZTBkLTEwN2JlOGE4ZTU1YSIsImlzcyI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MC9
        yZWFsbXMvYWRtaW5hcHBzIiwiYXVkIjoiYWNjb3VudCIsInN1YiI6IjhjOTY3YjZlLWZmYzUtNGQ1Yy1iYmJmLTlhNjRlODFlMjNjNyIsInR5cCI6IkJlYXJlciIsImF6cCI6InBob3RvLWFwcC1jb2RlLW
        Zsb3ctY2xpZW50Iiwic2lkIjoiWndhM3puRUw1Q2lDLVVWdjNLdFJqbmRhIiwiYWNyIjoiMSIsImFsbG93ZWQtb3JpZ2lucyI6WyIiXSwicmVhbG1fYWNjZXNzIjp7InJvbGVzIjpbIm9mZmxpbmVfYWNjZ
        XNzIiwiZGVmYXVsdC1yb2xlcy1hZG1pbmFwcHMiLCJkZXZlbG9wZXIiLCJ1bWFfYXV0aG9yaXphdGlvbiJdfSwicmVzb3VyY2VfYWNjZXNzIjp7ImFjY291bnQiOnsicm9sZXMiOlsibWFuYWdlLWFjY291
        bnQiLCJtYW5hZ2UtYWNjb3VudC1saW5rcyIsInZpZXctcHJvZmlsZSJdfX0sInNjb3BlIjoib3BlbmlkIG9yZ2FuaXphdGlvbiBtaWNyb3Byb2ZpbGUtand0IHBob25lIGVtYWlsIG9mZmxpbmVfYWNjZXN
        zIGFkZHJlc3MgcHJvZmlsZSIsInVwbiI6InZpYm9sbG9uZyIsImVtYWlsX3Zlcmlma WVkIjp0cnVlLCJuYW11IjoibG9uZyB2aWJvbCIsImdyb3VwcyI6WyJvZmZsaW5lX2FjY2VzcyIsImRlZmF1bHQtcm
        9sZXMtYWRtaW5hcHBzIiwiZGV2ZWxvcGVyIiwidW1hX2F1dGhvcml6YXRpb24iXSwicHJlZmVycmVkX3VzZXJuYW11Ijoidmlib2xsb25nIiwiZ2l2ZW5fbmFtZSI6Imxvbmci LCJmYW1pbHlfbmFtZSI6I
        nZpYm9sIiwiZW1haWwiOiJ2aWJvbGxvbmdAZ21haWwuY29tIn0.
        MJkaknQOhafxN6uKyb3G33zpJzhMCidLbhtLU1VS5cGntE506AOFGpVxU-GHkedj_Mfk9THxAxeB6GrGtAN-azpoqxc_woiv56bHQpA3Urns54-cGfjQDIL8T3QOkLvHH4u4DwhjvOWbFLm-ue31I08_fRg
        8tCSVBsqwD1AKwO3KsKks11N9KIxzpajP_m_vDyqtN7cpmhhbP374qQ0v7kNkmUz1DjUDIWQrxdPNUmE06UA3301d6VL9FITMqtBv5wlm94pvjQtjdPKA4UYpyJvBLeww-L5ZLAHuzs5_33J33h92q_wtsN
        tWUM9ZWeQ6tW6rYWcOua05hWKOoz-icQ",
```


- GetRequest directly from Client to the Resource Server port 8081

GET http://localhost:8081/users/status/check Send

Params Authorization Headers (8) Body Scripts Settings Cookies

Auth Type

Be...

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

Token 9ZWeQ6tW6rYWcOua05hWKOoz-icQ

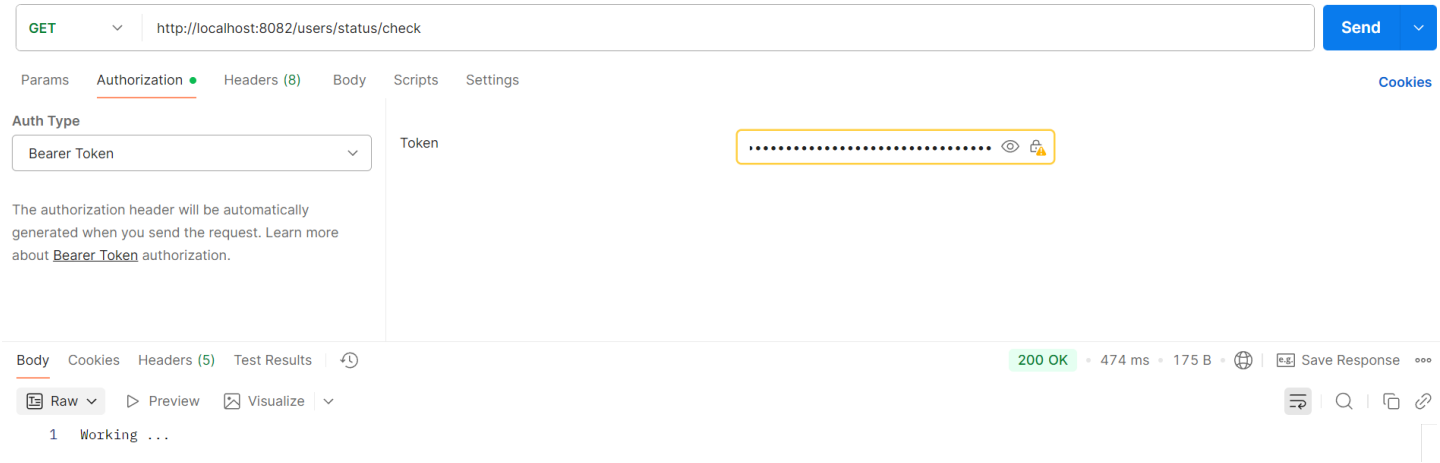
Body Cookies Headers (5) Test Results 200 OK 22 ms 175 B Save Response

Raw Preview Visualize

```
Working ...
```

GET: http://localhost:8081/users/status/check

- Request To ApiGateWay via port 8082



GET: http://localhost:8082/users/status/check